

Настройка и использование тулчейна Clang 7

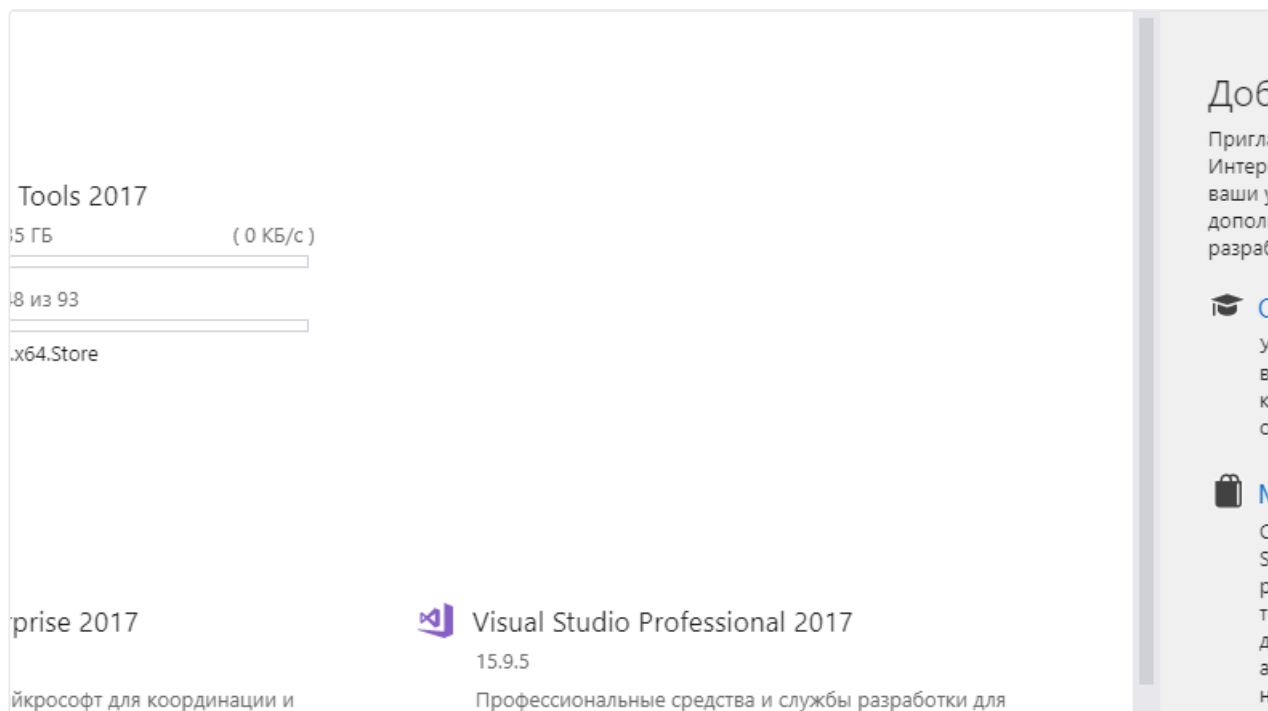
Windows

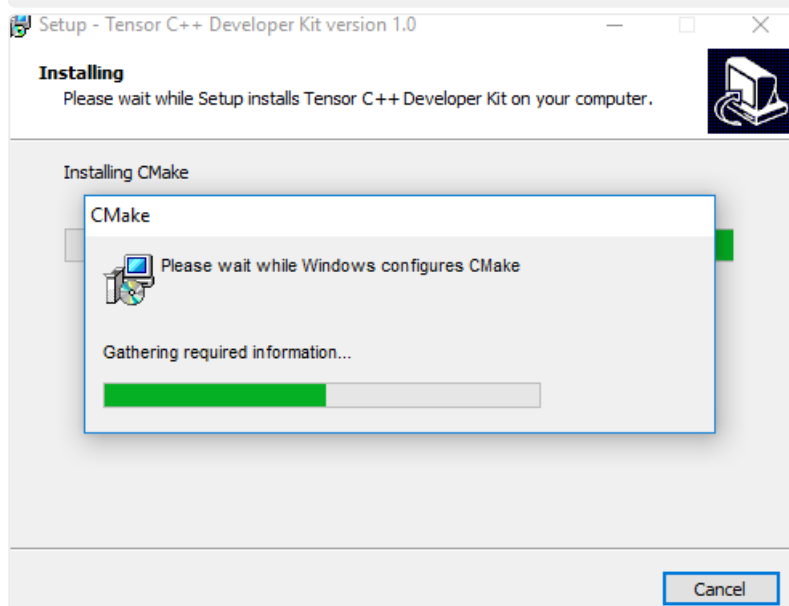
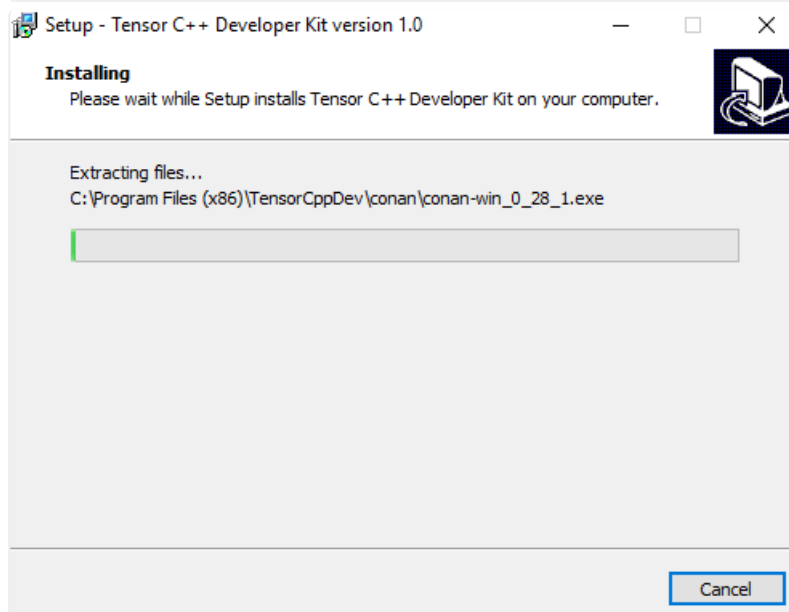
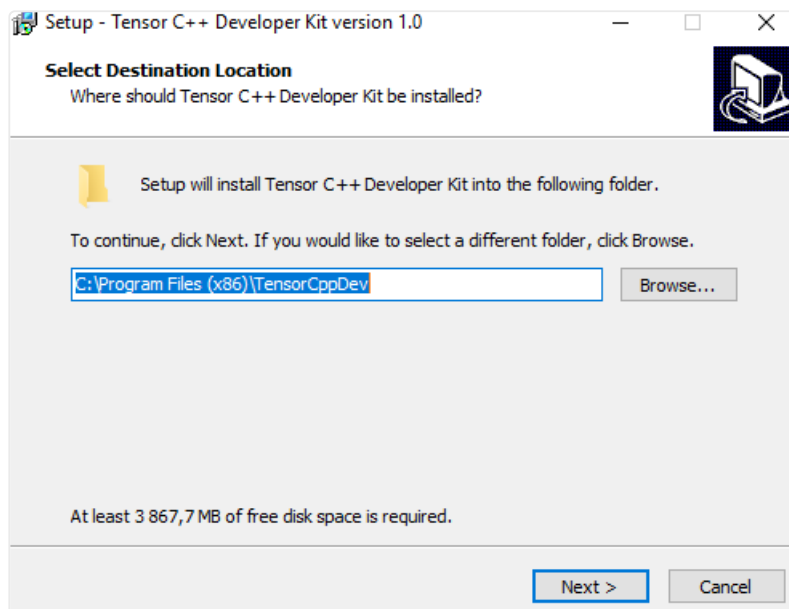
Установка TensorCppDev

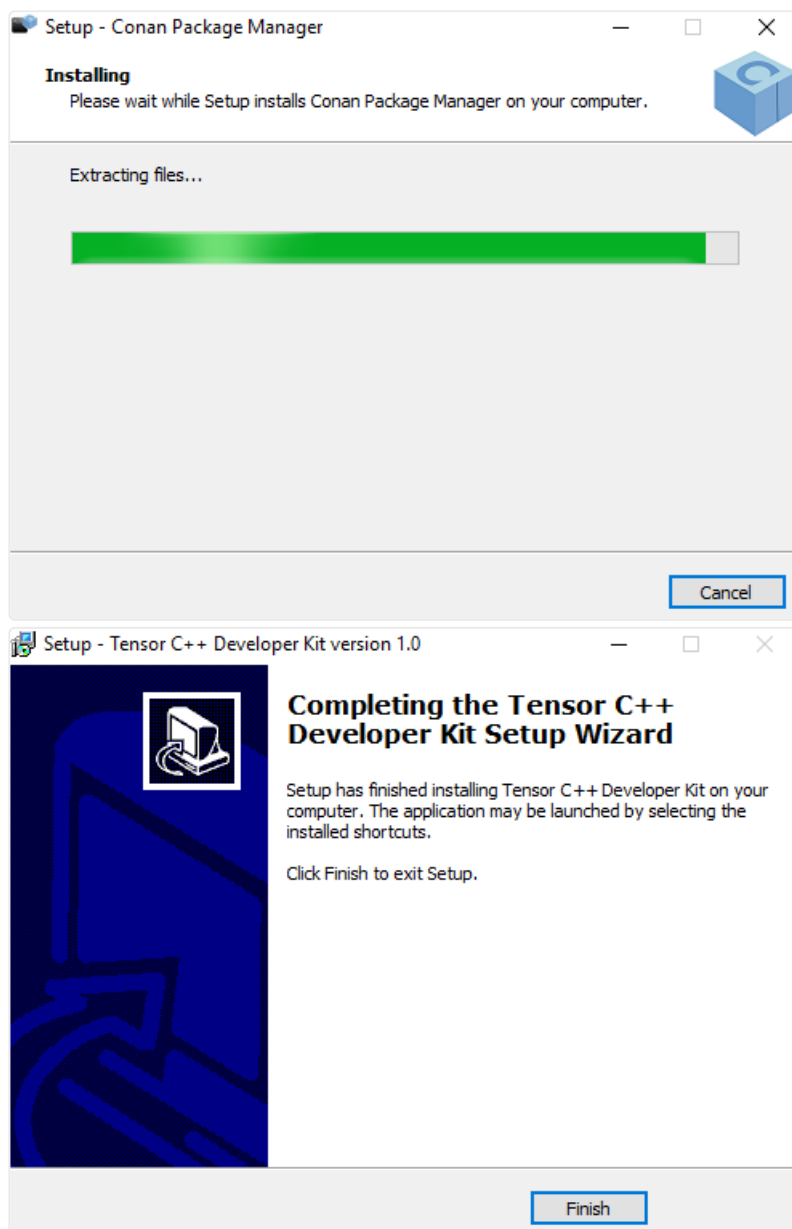
1. Скачайте **последнюю версию** инсталлятора TensorCppDev 1.x. На сетевом диске найдите директорию с инсталлятором последней версии, например, tensor-cpp-dev-1.6 и скопируйте её на локальный диск.

```
1 \\fs2.corp.tensor.ru\publicsoftware\Программное  
   обеспечение\Разработка\TensorCppDev\
```

2. Этот шаг выполняется только разработчиками СБИС Плагина: убедитесь, что на вашем ПК установлен [.Net Framework Developer Pack](#) версии 4.6 или старше. Подробнее в инструкции [СБИС3 Плагин. Сборка и развертывание](#).
3. Запустите установку TensorCppDev. Для этого запустите файл*.exe внутри директории инсталлятора. Например, tensor-cpp-dev-1.4.exe.
4. В процессе установки запустятся инсталляторы Visual Studio Build Tools (в процессе установки нельзя нажимать кнопку "Приостановить", иначе придется устанавливать заново), CMake, Conan и Ninja. Так устанавливаются совместимые версии утилит.





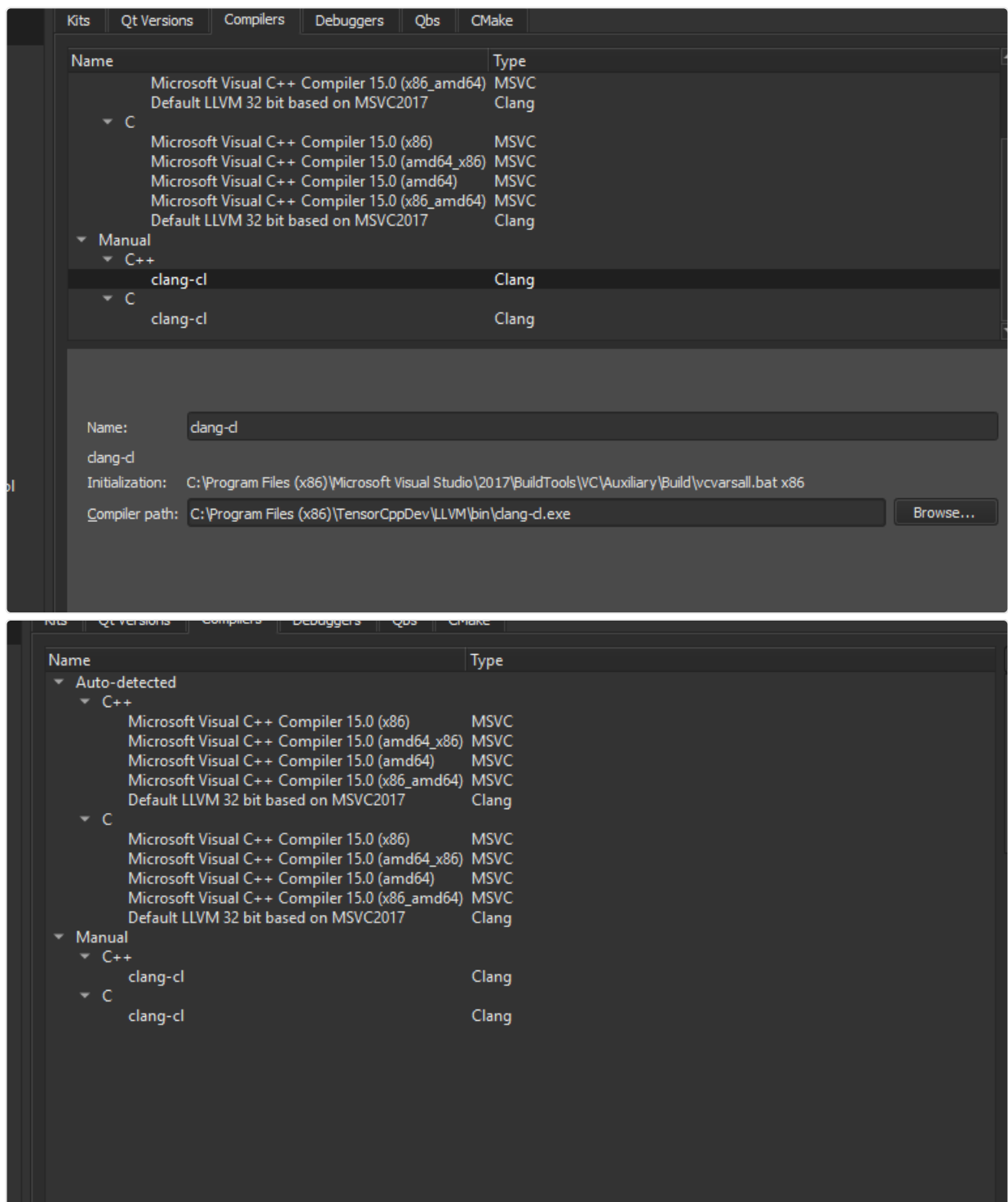


Для сборки 32-битного приложения запустите Tensor C++ Developer Command prompt.

Для сборки 64-битного приложения запустите Tensor C++ Developer Command prompt (x64).

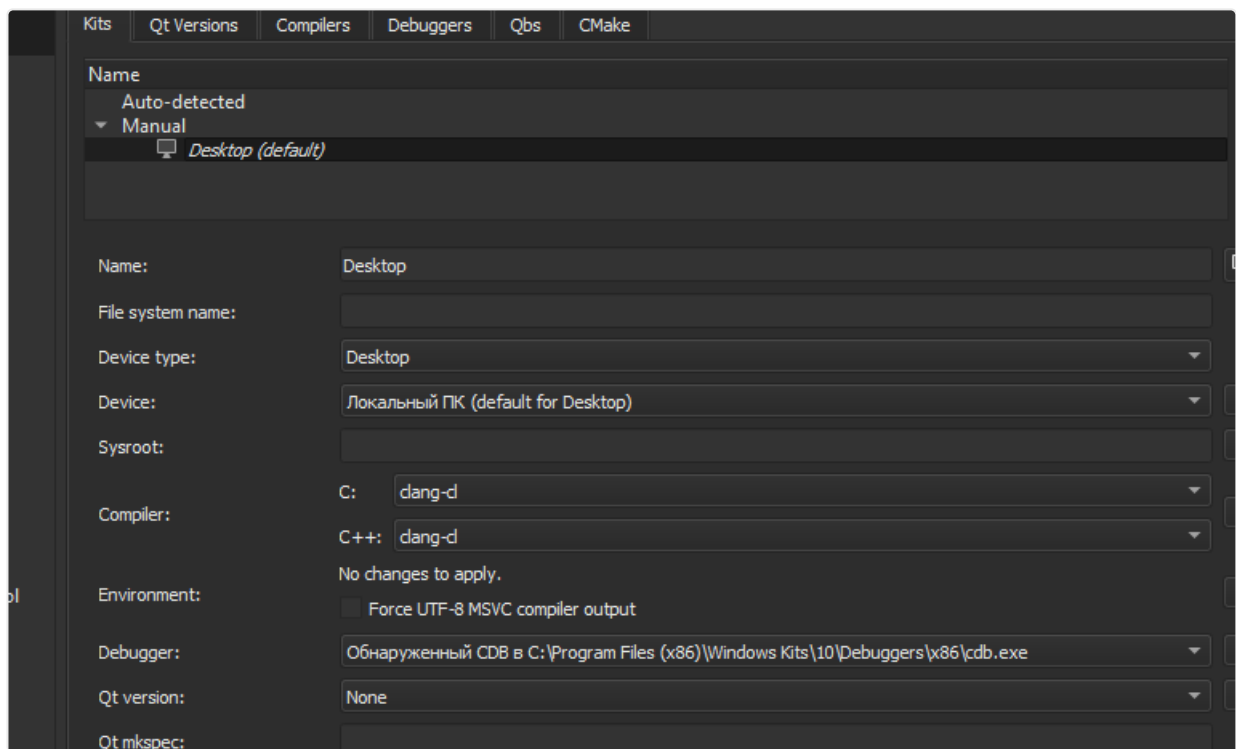
Добавление тулчейна

1. Установка Qt Creator подробно описана [здесь](#).
2. Запустите Qt Creator. Добавьте два компилятора clang-cl — для C и C++.
 - Перейдите в меню **Инструменты (Tools)** → **Параметры... (Options)** → пункт меню **Комплекты (Kits)** → вкладка **Компиляторы (Compilers)**.
 - по кнопке "Добавить (Add)" добавьте clang-cl → C.
 - в атрибуте **Инициализация (Initialization)** выберите C:\Program Files (x86)\Microsoft Visual Studio\2017\BuildTools\VC\Auxiliary\Build\vcvarsall.bat x86.
 - в атрибуте **Путь к компилятору (Compiler path)** укажите путь до файла clang-cl, например, C:\Program Files (x86)\TensorCppDev\LLVM\bin\clang-cl.Настройка clang-cl при добавлении для C++ выполняется аналогично.



3. Выполните создание комплекта (Kit).

- Перейдите в меню **Инструменты (Tools)** → **Параметры... (Options)** → пункт меню **Комплекты (Kits)** → вкладка **Kits (Комплекты)**.
- В атрибуте **Имя (Name)** установите произвольное имя комплекта.
Только для разработчиков мобильного приложения: для удобства использования рекомендуется выбрать имя, которое будет соответствовать архитектуре, под которую собирается проект мобильного приложения.
- В атрибуте **Компилятор (Compiler)** укажите:
 - для C — clang-cl.
 - для C++ — clang-cl.
- В атрибуте **Отладчик (Debugger)** укажите "CDB".
- В атрибуте **Генератор CMake (CMake Generator)** укажите "CodeBlocks — Ninja".



- В атрибуте **Среда (Environment)** добавьте путь до Python 3.7, например:

```
1 PATH+=C:\Program Files (x86)\Python37-32;C:\Program Files
  (x86)\Python37-32\Scripts
```

CentOS

Установка метапакетов

Для того чтобы получить тулчейн на базе Clang 7, потребуется [установить/обновить стандартные мета-пакеты](#), которые должны быть установлены у каждого разработчика: `sbis-development` и `sbis-development-tools`.

После установки пакетов выполните последовательно команды:

```
1 sudo -i
2 ln -s -v /opt/python3.7/bin/python3.7 /usr/bin/python3.7
```

и перезагрузите компьютер.

Сборочный тулчейн появится под директорией `/opt`. Каждая из частей тулчейна устанавливается в отдельную директорию:

```
1 [root@centos-build-vm /]# ls -l /opt
2 total 7
3 drwxr-xr-x  5 root root  40 июл  8 15:25 clang-7.0.1
4 drwxr-xr-x  8 root root  78 июл  8 15:25 gcc-9.1.1
5 drwxr-xr-x  4 root root  28 июл  8 15:25 lld-7.0.1
6 drwxr-xr-x  3 root root  18 июл  8 15:25 llvm-7.0.1
7 drwxr-xr-x  1 root root  38 июл  8 15:25 python3.7
```

Сборка через консоль

Для того чтобы воспользоваться тулчейном, достаточно определить несколько переменных окружения, которые подскажут CMake каким окружением следует воспользоваться для конфигурирования решений:

```
1 # CMake проверяет значения этих двух переменных окружения для
  определения компилятора.
```

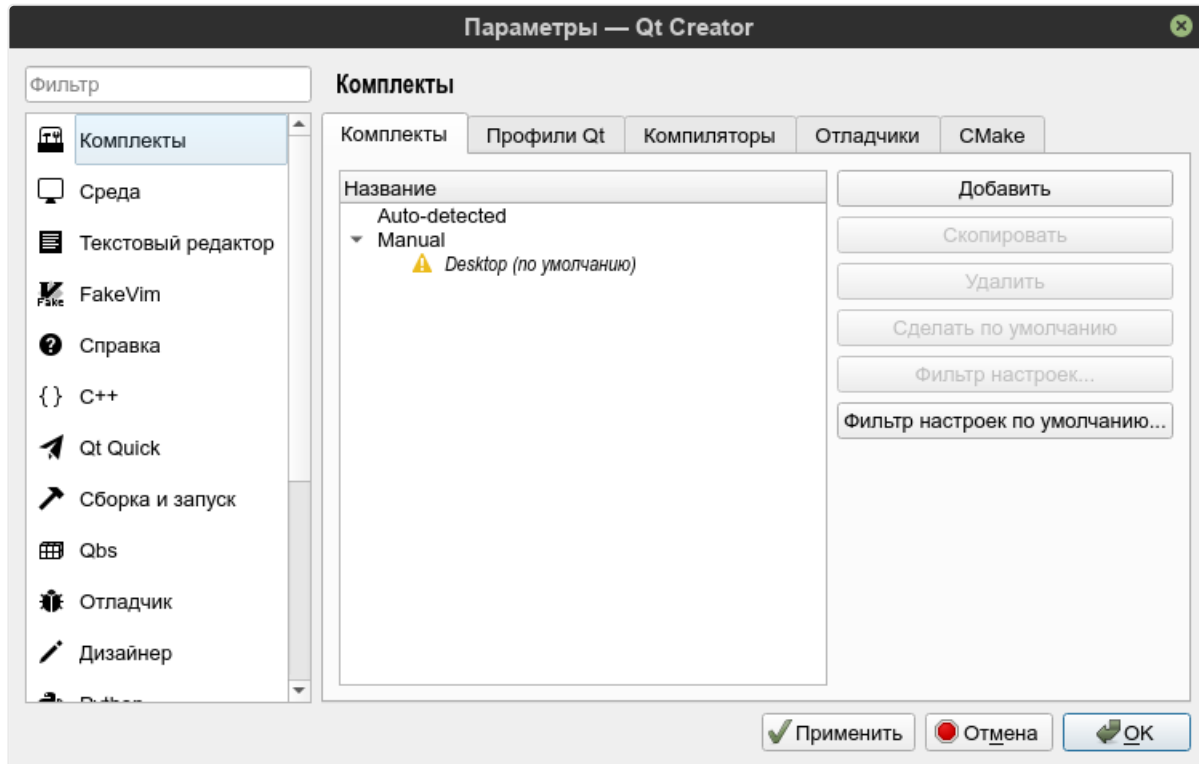
```

2 #
3 # Также можно в качестве альтернативы указать
4 # при вызове CMake -DCMAKE_C_COMPILER и -DCMAKE_CXX_COMPILER
5 export CC=/opt/clang-7.0.1/bin/clang
6 export CXX=/opt/clang-7.0.1/bin/clang++
7 # Подставляем в PATH пути поиска до python3.7 и ld.lld
8 export PATH=/opt/python3.7/bin:/opt/ld-7.0.1/bin:${PATH}

```

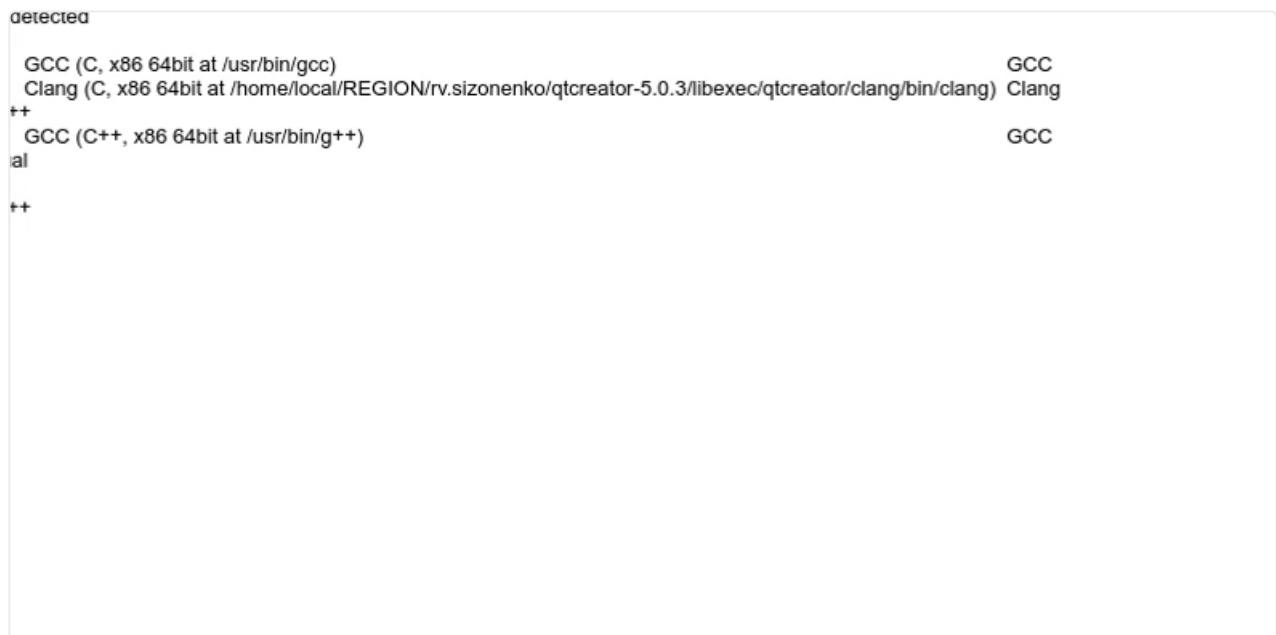
Настройка комплекта

Перейдите на вкладку **Инструменты** → **Параметры** → **Комплекты**, откроется окно следующего вида:



Создайте соответствующие записи для компиляторов C и C++, чтобы использовать их в комплекте.

1. Перейдите на вкладку **Компиляторы**.
2. С помощью кнопки "Добавить" выберите соответствующие компиляторы для C и C++, прописав пути до бинарных файлов, расположенных в /opt/clang-7.0.1/bin.



звание:	Custom Clang 7.0.1									
ть к компилятору:	/opt/clang-7.0.1/bin/clang									
аги генерации кода для платформы:										
аги компоновки для платформы:										
I:	x86-linux-generic-	x86	-	linux	-	generic	-	elf	-	64bit

вание:	Custom Clang++ 7.0.1									
ь к компилятору:	/opt/clang-7.0.1/bin/clang++									
аги генерации кода для платформы:										
аги компоновки для платформы:										
:	x86-linux-generic-	x86	-	linux	-	generic	-	elf	-	64bit

Для добавления отладчика выполните следующие шаги:

1. Перейдите на вкладку **Отладчики**.

Система GDB в /bin/gdb	/bin/gdb	GDB
qual		

2. С помощью кнопки "Добавить" выберите gdb-отладчик, прописав пути до бинарного файла, расположенного в /opt/rh/devtoolset-9/root/bin/gdb.

Система GDB в /bin/gdb	/bin/gdb	GDB
Нужно	Новый отладчик	Не определён
Имя: Новый отладчик Путь: Команда: Не определён		

Система GDB в /bin/gdb	/bin/gdb	GDB
Нужно	gdb83	/opt/rh/devtoolset-9/root/bin/gdb GDB
Имя: gdb83 Путь: /opt/rh/devtoolset-9/root/bin/gdb Команда: GDB		



Для CentOS 8 выберите gdb-отладчик, прописав пути до бинарного файла, расположенного в /opt/rh/gcc-toolset-9/root/bin/gdb.

Для создания комплекта с настроенным компилятором и отладчиком выполните следующие шаги:

1. Перейдите на вкладку **Комплекты**.
2. Создайте новый комплект с помощью кнопки "Добавить".
3. В качестве компиляторов C и C++ укажите только что созданные записи для Clang 7.
4. В качестве отладчика укажите только что созданную запись для GDB.

Также необходимо добавить в окружение пути до Python 3.7 и LLD:

1. На вкладке **Комплекты** нажмите кнопку "Изменить" напротив пункта **Среда**.

o-detected
nual

⚠ Desktop (по умолчанию)

⚠ Без имени

файловой системе:

устройства:

йство:

evice:

ь образа:

C:

C++:

2. Добавьте переменные CC, CXX, а также соответствующие пути в PATH:

```
1 CC=/opt/clang-7.0.1/bin/clang
2 CXX=/opt/clang-7.0.1/bin/clang++
3 PATH=/opt/python3.7/bin:/opt/llvm-7.0.1/bin:${PATH}
```

Изменение среды — Qt Creator

CC=/opt/clang-7.0.1/bin/clang
CXX=/opt/clang-7.0.1/bin/clang++
PATH=/opt/python3.7/bin:/opt/llvm-7.0.1/bin:\${PATH}

Enter one environment variable per line.
To set or change a variable, use VARIABLE=VALUE.
To append to a variable, use VARIABLE+=VALUE.
To prepend to a variable, use VARIABLE+=VALUE.
Existing variables can be referenced in a VALUE with \${OTHER}.
To clear a variable, put its name on a line with nothing else on it.
To disable a variable, prefix the line with "#".

⚠ Desktop (по умолчанию)

⚠ Custom Clang Tool

файловой системе:

устройства:

йство:

evice:

ь образа:

C:

C++:

C:

чик:

иль Qt:

